Group Topic: **Big Data Analysis for Rainfall-Based Flood Risk Prediction in Albay using SYNOP Data** | Week 2 Task

Git Repo/Colab Link: https://github.com/PotatoKevz/About-bda | Date: 4/24/2026

---

**Objectives**

The goal for this week was to gather and prepare our SYNOP weather data from the OGIMET website. We needed to scrape the raw SYNOP reports, decode them, and clean up the data so it's ready for analysis. We also had to make sure we hit the 500,000-row minimum since our project is focused on rainfall-based flood risk prediction in Albay.

**Tasks Completed**

Write here the list of things you accomplished this week (if any), and with screenshots or pictures. Describe the steps you followed. Include algorithms, data flow, or specific Spark transformations/actions if applicable. Use numbered or bullet points for clarity.

1. Data Gathering – We found that OGIMET (ogimet.com) has historical SYNOP observations from weather stations all over the world, including ones near Albay. We identified the relevant stations and figured out how to pull data from the site.
2. Method of Data Procurement – We wrote a Python script that scrapes OGIMET by sending HTTP requests for specific stations and date ranges. It downloads the raw SYNOP-encoded messages in bulk, which saves a lot of time compared to doing it manually.
3. Data Cleaning / Data Processing – We also made a separate Python script to decode the SYNOP codes. The raw messages use a WMO format where things like temperature, wind, pressure, and rainfall are all packed into these short code groups. Our script parses through them and pulls out the actual values into a proper table format.
4. Dataset Validation – We checked that the data we collected clears the 500,000-row requirement. The dataset can be split into multiple files if needed, and since OGIMET is a public source, we're good on the data ethics side.

**Tasks to be Completed**

- Complete all assigned task/s for the following week.

**Results and Analysis**

The data gathering phase successfully demonstrated that SYNOP meteorological reports can be retrieved from the OGIMET website through automated web scraping. The scraper script was able to retrieve raw SYNOP-coded messages for the target stations. The SYNOP decoder script was able to parse and extract key meteorological variables including temperature, dew point, wind speed, wind direction, atmospheric pressure, visibility, and precipitation indicators from the encoded messages. Initial tests confirm the feasibility of accumulating more than 500,000 rows across multiple stations and time periods, satisfying the minimum data requirement for the project.

C: > Users > romeb > Desktop > Big Data Analysis > Programs > flood-prediction > ogimet_synop_scraper.py > _chunks

```python
BASE_URL = "http://www.ogimet.com/cgi-bin/getsynop"
COLUMNS  = ["WMOIND", "YEAR", "MONTH", "DAY", "HOUR", "MIN", "REPORT"]

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
log = logging.getLogger(__name__)

# Mimic a real browser — OGIMET rejects bot-like user agents
HTTP_HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/124.0.0.0 Safari/537.36"
    ),
    "Accept"          : "text/plain, text/html, */*",
    "Accept-Language" : "en-US,en;q=0.9",
    "Connection"      : "keep-alive",
}


# — helpers ————————————————————————————————————————————————————————

def _fmt(dt: datetime) -> str:
    return dt.strftime("%Y%m%d%H%M")


def _chunks(start_year: int, end_year: int, days: int) -> Iterator[Tuple[datetime, datetime]]:
    """Yield (begin, end) datetime pairs in `days`-sized steps."""
    now   = datetime.now(timezone.utc).replace(tzinfo=None)
```

**Python Scraper Script**

C: > Users > romeb > Desktop > Big Data Analysis > Programs > flood-prediction > synop_decoder.py > ...

```python
def decode_irix(group: str) -> dict:
    """Section 0 indicator group  iRiXhVV  (5 chars).

    iR (precipitation indicator):
        0/1/2 — group 6 present somewhere  → no sentinel (use decoded value)
        3     — precip amount = 0, no group 6  → rain_sentinel = 0.0  (Case B)
        4     — station does not measure precip → rain_sentinel = None (Case A)
    """
    out = {}
    if len(group) != 5:
        return out
    ir = group[0]
    if ir == "3":
        out["rain_sentinel"] = 0.0   # "no rain reported" → 0.0 mm
    elif ir == "4":
        out["rain_sentinel"] = None  # truly missing (station doesn't measure)
    # iX - station type / present-weather indicator  (ignored)
    # h  - height of cloud base (code table 1600)
    h_code = group[2]
    h_table = {"0": 0, "1": 50, "2": 100, "3": 200, "4": 300,
               "5": 600, "6": 1000, "7": 1500, "8": 2000, "9": 2500, "/": None}
    out["cloud_base_m"] = h_table.get(h_code)
    # VV - visibility (code table 4377)
    vv = group[3:5]
    out["visibility_m"] = _decode_vv(vv)
    return out


def _decode_vv(vv: str) -> float | None:
    """Convert VV code (00-99) to metres (WMO Code Table 4377)."""
```

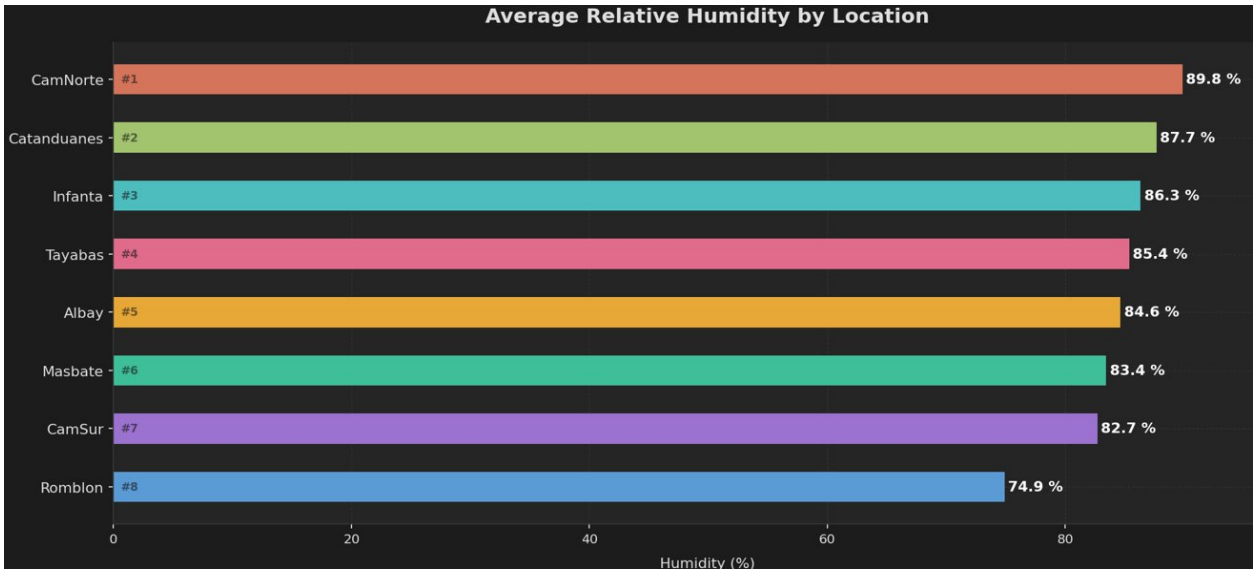
**Python Decoder Script**

| WMOIND | YEAR | MONTH | DAY | HOUR | MIN | REPORT |
|---|---|---|---|---|---|---|
| 98444 | 2000 | 1 | 1 | 0 | 0 | AAXX 01001 98444 11460 80403 10248 20246 40117 52022 60514 78166 83270 333 20248 56999 58001 70506 83818 88459 555 20470= |
| 98444 | 2000 | 1 | 1 | 3 | 0 | AAXX 01031 98444 21462 80402 10262 20249 40111 58008 72582 83278 333 56999 60037 83818 85259 88270= |
| 98444 | 2000 | 1 | 1 | 6 | 0 | AAXX 01061 98444 11465 60505 10280 20256 40085 57026 60091 70182 83278 333 56999 83818 86270= |
| 98444 | 2000 | 1 | 1 | 9 | 0 | AAXX 01091 98444 32465 60904 10282 20253 40087 53002 82272 333 56999 82818 83459 85070= |
| 98444 | 2000 | 1 | 1 | 12 | 0 | AAXX 01121 98444 32465 60404 10271 20246 40112 52025 82202 333 10290 56999 58004 82818 85070= |
| 98444 | 2000 | 1 | 1 | 15 | 0 | AAXX 01151 98444 32465 60402 10254 20243 40115 50003 82278 333 56999 82818 86270= |
| 98444 | 2000 | 1 | 1 | 18 | 0 | AAXX 01181 98444 32465 50403 10262 20229 40094 57021 82208 333 56999 82818 85070= |
| 98444 | 2000 | 1 | 2 | 0 | 0 | AAXX 02001 98444 11465 70702 10257 2038/ 40114 52024 60104 70282 83238 333 20248 56999 59003 70110 82818 84359 83270 555 20010= |
| 98444 | 2000 | 1 | 2 | 3 | 0 | AAXX 02031 98444 21468 70402 10292 20244 40117 50003 72582 82338 333 56999 69907 82818 83459 84270= |
| 98444 | 2000 | 1 | 2 | 6 | 0 | AAXX 02061 98444 11460 70901 10255 20247 40096 57021 60281 78098 84378 333 56999 82918 84459 87270= |
| 98444 | 2000 | 1 | 2 | 9 | 0 | AAXX 02091 98444 21465 70402 10268 20247 40090 55006 70282 83372 333 56999 69907 83918 84459 86070= |
| 98444 | 2000 | 1 | 2 | 12 | 0 | AAXX 02121 98444 32465 50702 10256 20240 40110 53020 82272 333 10292 5699 59002 82818 83070= |
| 98444 | 2000 | 1 | 2 | 15 | 0 | AAXX 02151 98444 32465 40402 10250 20234 40117 50007 82272 333 56999 82818 83070= |
| 98444 | 2000 | 1 | 2 | 18 | 0 | AAXX 02181 98444 32465 60502 10252 20236 40100 58017 83278= |
| 98444 | 2000 | 1 | 2 | 21 | 0 | AAXX 02211 98444 21462 80403 10240 20235 40097 55003 78082 8427/ 333 56999 60107 84818 83359= |
| 98444 | 2000 | 1 | 3 | 0 | 0 | AAXX 03001 98444 11468 80404 10248 20236 40118 52021 60384 72582 84377 333 20240 56999 58004 70382 82918 83459 88270 555 20102= |
| 98444 | 2000 | 1 | 3 | 3 | 0 | AAXX 03031 98444 21465 70403 10258 20243 40120 50002 72562 83378 333 56999 60047 81918 84459 85270= |
| 98444 | 2000 | 1 | 3 | 6 | 0 | AAXX 03061 98444 11468 70403 10270 20243 40097 57023 60041 72582 83378 333 56999 82918 84459 85270= |
| 98444 | 2000 | 1 | 3 | 9 | 0 | AAXX 03091 98444 21465 50404 10264 20245 40096 55001 70182 82232 333 56999 69947 82818 83070= |
| 98444 | 2000 | 1 | 3 | 12 | 0 | AAXX 03121 98444 11465 60403 10250 20231 40114 52018 69931 71522 82208 333 10281 56999 59001 82818 86270= |
| 98444 | 2000 | 1 | 4 | 0 | 0 | AAXX 04001 98444 12468 50402 10260 20230 40124 52024 60044 83308 333 20242 56999 58006 70042 81918 82820 84270= |
| 98444 | 2000 | 1 | 4 | 3 | 0 | AAXX 04031 98444 32468 50405 10289 20238 40127 50006 83402 333 56999 82918= |
| 98444 | 2000 | 1 | 4 | 6 | 0 | AAXX 04061 98444 11468 60203 10293 20241 40101 57026 69901 72582 84232 333 56999 82915= |
| 98444 | 2000 | 1 | 4 | 9 | 0 | AAXX 04091 98444 21465 70405 10264 20249 40097 55004 78022 85302 333 56999 60017 83918 82820 83459 85070= |
| 98444 | 2000 | 1 | 4 | 12 | 0 | AAXX 04121 98444 11465 60403 10251 20233 40126 52029 60011 71522 84372 333 10298 56999 58012 81918 83820 83459= |
| 98444 | 2000 | 1 | 4 | 15 | 0 | AAXX 04151 98444 32465 40402 10251 20236 40135 51009 83202 333 56999 83818 83070= |
| 98444 | 2000 | 1 | 4 | 18 | 0 | AAXX 04181 98444 32465 30403 10249 20234 40117 57018 83202= |

**Undecoded SYNOP Reports**

| station_id | date | time | lat | lon | temp | pressure | humidity | wind_speed | wind_dir | cloud_cove | visibility_m | rain_3h |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 98444 | 1/1/2000 | 0:00 | 13.15 | 123.73 | 24.8 | 1011.7 | 98.8 | 3 | 40 | 8 | 15000 | 51.0 |
| 98444 | 1/1/2000 | 3:00 | 13.15 | 123.73 | 26.2 | 1011.1 | 92.6 | 2 | 40 | 8 | 17000 | 3.0 |
| 98444 | 1/1/2000 | 6:00 | 13.15 | 123.73 | 28.0 | 1008.5 | 86.8 | 5 | 50 | 6 | 20000 | 9.0 |
| 98444 | 1/1/2000 | 9:00 | 13.15 | 123.73 | 28.2 | 1008.7 | 84.3 | 4 | 90 | 6 | 20000 | 0.0 |
| 98444 | 1/1/2000 | 12:00 | 13.15 | 123.73 | 27.1 | 1011.2 | 86.2 | 4 | 40 | 6 | 20000 | 0.0 |
| 98444 | 1/1/2000 | 15:00 | 13.15 | 123.73 | 25.4 | 1011.5 | 93.6 | 2 | 40 | 6 | 20000 | 0.0 |
| 98444 | 1/1/2000 | 18:00 | 13.15 | 123.73 | 26.2 | 1009.4 | 82.1 | 3 | 40 | 5 | 20000 | 0.0 |
| 98444 | 1/2/2000 | 0:00 | 13.15 | 123.73 | 25.7 | 1011.4 | NaN | 2 | 70 | 7 | 20000 | 10.0 |
| 98444 | 1/2/2000 | 3:00 | 13.15 | 123.73 | 29.2 | 1011.7 | 75.4 | 2 | 40 | 7 | 23000 | 0.1 |
| 98444 | 1/2/2000 | 6:00 | 13.15 | 123.73 | 25.5 | 1009.6 | 95.3 | 1 | 90 | 7 | 15000 | 28.0 |
| 98444 | 1/2/2000 | 9:00 | 13.15 | 123.73 | 26.8 | 1009.0 | 88.3 | 2 | 40 | 7 | 20000 | 0.1 |
| 98444 | 1/2/2000 | 12:00 | 13.15 | 123.73 | 25.6 | 1011.0 | 90.9 | 2 | 70 | 5 | 20000 | 0.0 |
| 98444 | 1/2/2000 | 15:00 | 13.15 | 123.73 | 25.0 | 1011.7 | 90.8 | 2 | 40 | 4 | 20000 | 0.0 |
| 98444 | 1/2/2000 | 18:00 | 13.15 | 123.73 | 25.2 | 1010.0 | 90.9 | 2 | 50 | 6 | 20000 | 0.0 |
| 98444 | 1/2/2000 | 21:00 | 13.15 | 123.73 | 24.0 | 1009.7 | 97.0 | 3 | 40 | 8 | 17000 | 10.0 |
| 98444 | 1/3/2000 | 0:00 | 13.15 | 123.73 | 24.8 | 1011.8 | 93.1 | 4 | 40 | 8 | 23000 | 38.0 |
| 98444 | 1/3/2000 | 3:00 | 13.15 | 123.73 | 25.8 | 1012.0 | 91.4 | 3 | 40 | 7 | 20000 | 4.0 |
| 98444 | 1/3/2000 | 6:00 | 13.15 | 123.73 | 27.0 | 1009.7 | 85.2 | 3 | 40 | 7 | 23000 | 4.0 |
| 98444 | 1/3/2000 | 9:00 | 13.15 | 123.73 | 26.4 | 1009.6 | 89.3 | 4 | 40 | 5 | 20000 | 0.4 |
| 98444 | 1/3/2000 | 12:00 | 13.15 | 123.73 | 25.0 | 1011.4 | 89.2 | 3 | 40 | 6 | 20000 | 0.3 |
| 98444 | 1/4/2000 | 0:00 | 13.15 | 123.73 | 26.0 | 1012.4 | 83.6 | 2 | 40 | 5 | 23000 | 4.0 |
| 98444 | 1/4/2000 | 3:00 | 13.15 | 123.73 | 28.9 | 1012.7 | 74.0 | 5 | 40 | 5 | 23000 | 0.0 |
| 98444 | 1/4/2000 | 6:00 | 13.15 | 123.73 | 29.3 | 1010.1 | 73.6 | 3 | 20 | 6 | 23000 | 0.1 |

**Decoded SYNOP Reports**




**One of Many Visualizations**

**Challenges and Solutions**

We ran into a few issues along the way, but we were able to work through them:

Challenge 1: SYNOP Report Cleaning and Formatting – The raw SYNOP messages aren't exactly easy to work with. They're densely coded, and there's a lot of inconsistency – missing groups, weird spacing, malformed records. It made parsing pretty tricky at first.

Solution: We updated the decoder script to handle these cases more gracefully – if a code group is malformed or missing, it just skips that part and still pulls whatever valid data is there.

Challenge 2: Getting Enough Rows – One station over a short time range doesn't come close to 500,000 rows, so we had to think about how to scale up the dataset.

Solution: We expanded the scraper to pull from multiple stations across the Bicol region and stretched the date range to cover several years. Combining all of that together gets us to the required volume.

Challenge 3: Getting the Data from OGIMET – OGIMET doesn't have a proper download API, so everything has to go through HTTP requests. We also ran into issues with rate limiting and the site sometimes not returning what we expected.

Solution: We added throttling and retry logic to the scraper so it doesn't hit the site too hard and can recover from failed requests. We also set it up so it can pick up where it left off instead of starting from scratch every time.

**Conclusion**

Overall, we got a lot done this week. We built a scraper to pull SYNOP reports from OGIMET and a decoder to turn those reports into usable data. It wasn't smooth sailing – the SYNOP format is messy, getting to 500k rows took some work, and scraping OGIMET itself had its quirks – but we found solutions

for all of it. We're now confident the data pipeline works and we have a solid base to move into the analysis and modeling phase.